1 Software

The software subsystem, running on the central Adafruit Metro M0 microcontroller, serves as the overarching "brain" of the EEELunarRover. Because the software team does not have direct control over the analogue circuit topologies, the digital architecture was designed to take the digitised signals from the four hardware front- ends, logically classify the rock properties, command the mecanum drive system, and present the final outcome to the operator.

To ensure a highly reliable and responsive system during the final demo arena test, four measurable requirements were established for this subsystem:

1. Age Decoding: Parse the 600 bps UART radio signal with a frame error rate of less than 1%.
1. IR Classification: Accurately classify the Poisson-distributed infrared pulses ($\lambda = 312$ vs 547 s^{-1}) with >95% accuracy within a 200 ms polling window.
1. Fusion Robustness: Maintain 100% correct rock classification even in the event of a single hardware sensor failure.
1. Dashboard Latency: Host a remote web dashboard over WiFi with an end-to- end telemetry latency of less than 500 ms.

1.1 Control System

The rover control system was implemented as a four-motor mecanum-wheel drive, enabling translational motion in the forward, reverse, left, right and diagonal directions, as well as in-place rotation. Each motor is driven using one digital direction signal and one PWM speed signal. The final pin allocation was selected to avoid conflicts with the WiFi shield, particularly the reserved pins D5, D7 and D10.

The following pin connections are being made:

Driver	**Motor**	**Direction pin**	**PWM pin**	**Note**
D1	M_{FL}	D8	D11	
D1	M_{FR}	D13	D12	Enable reassigned from D7
D2	M_{RL}	D4	D6	
D2	M_{RR}	D2	D3	Direction reassigned from D5

Table 1: Final motor-driver pin allocation after resolving WiFi-shield pin conflicts.

1.1.1 Control Logic

Motor control is handled by a common drive routine that assigns signed speed directions to the four wheels, where +1 represents forward rotation, −1 represents reverse rotation and 0 represents a stopped motor. This abstraction allows each rover movement to be described as a four-element wheel command vector:

$$[M_{\mathrm{FL}}, M_{\mathrm{FR}}, M_{\mathrm{RL}}, M_{\mathrm{RR}}]\,.$$

The implemented mecanum-drive mapping is shown in Table 2.

Command	M_{FL}	M_{FR}	M_{RL}	M_{RR}
Forward	$+1$	$+1$	$+1$	$+1$
Backward	-1	-1	-1	-1
Left	-1	$+1$	$+1$	-1
Right	$+1$	-1	-1	$+1$
Forward-left	0	$+1$	$+1$	0
Forward-right	$+1$	0	0	$+1$
Backward-left	-1	0	0	-1
Backward-right	0	-1	-1	0
Rotate left	-1	$+1$	-1	$+1$
Rotate right	$+1$	-1	$+1$	-1
Stop	0	0	0	0

Table 2: Mecanum-wheel motor direction mapping used by the rover control system.

The user interface was implemented as a browser-based dashboard communicating with the rover through HTTP requests. Keyboard control was integrated using the keys W, A, S, D, Q and E. Single-key commands provide the cardinal movements and rotation, while two-key combinations provide diagonal movement. The final command scheme is:

`w` : forward, `s` : backward, `a` : left, `d` : right,
`wa` : forward-left, `wd` : forward-right, `sa` : backward-left, `sd` : backward-right,
`q` : rotate left, `e` : rotate right.

A safety-stop behaviour was added to the control logic. If no key is pressed, if the browser window loses focus, or if an invalid key combination is detected, the dashboard sends a stop command to the rover. The rover-side command handler also treats empty or unknown command strings as a stop condition. This provides protection against continued motion if a command is interrupted, released or corrupted during transmission.

Evaluation was carried out by checking that each dashboard input produced the expected HTTP motor command and corresponding wheel-vector output. The movement commands were verified against the mecanum-drive mapping in Table 2. The stop condition was also tested by releasing all keys, pressing invalid key combinations such as opposing directions, and switching browser focus. In each case, the rover command state returned to stop, confirming that the control interface fails safely under normal user-input and communication-edge cases.

1.1.2 Variable Speed

To improve controllability across different driving conditions, three discrete speed modes were implemented in the dashboard control system. The motor driver already accepts a PWM speed value in the range 0–255, so the dashboard was modified to select and transmit a predefined PWM value with each movement command. This allows the rover speed to be adjusted without changing the underlying motor-control logic.

The implemented speed modes are shown in Table 3.

Mode	Duty Cycle	PWM value	Intended use
Long-range navigation	100%	255	Fast movement across open areas
Narrow-gap navigation	75%	191	Controlled movement through tighter spaces
Close sensing	25%	64	Slow approach when collecting sensor readings

Table 3: Implemented variable speed modes for rover movement control.

Each dashboard movement command is sent to the rover as an HTTP request containing both the direction command and the selected speed value. The rover firmware then updates the global motor speed from the received `speed` parameter before executing the requested wheel-motion command.

This approach separates speed selection from direction control. The direction logic remains based on the mecanum wheel command vectors, while the speed mode determines the PWM magnitude applied to all active motors. Motors assigned a zero value in the command vector remain stopped, while active motors receive the selected PWM value.

The speed modes were evaluated by confirming that each dashboard mode generated the correct PWM value and that the rover responded with visibly different motion speeds. The 100% mode provided maximum travel speed for open-area navigation, the 75% mode gave improved controllability in constrained paths, and the 25% mode allowed slower, more precise positioning during close-range sensing. This confirmed that the discrete speed selection improved usability without adding complexity to the firmware-side movement mapping.